SESSION 04 / PRINTABLE GUIDE

Lesson 3: Clean device events and publish daily summaries

Session 04 | Worked example 2
Output: Raw/accepted/quarantined events and a two-row device/day transfer table.

Micro-fixture track: These nine events explain individual rules. Use the main lab for 201,579 readings, fourteen days of patterns, site/type context, coverage analysis, and 700 device/day summaries. Do not use the assertions or Warehouse scripts from this small track for that tier.

Why use this example?

File-oriented telemetry often needs reusable timestamp, event-key, range, and quality rules before it becomes a reporting fact. A notebook can combine those checks with distributed aggregation and later specialized feature engineering.

This nine-row fixture is deliberately small enough to explain. It is not a Spark performance benchmark, a streaming solution, or proof that equivalent SQL would be inferior.

Prepare the input

If using this optional micro track, pre-stage its input **before the delivery day** as part of the prerequisite gate.

Use the isolated default Lakehouse from the notebook preflight. Upload device_readings.csv to `Files/session_04/devices/batch-001/device_readings.csv`.

Use the current nine-column fixture, replacing an earlier four-column copy if present; do not add a second active file. The schema-overwrite options below deliberately update only the named isolated teaching tables to this revised contract. They are not a production schema-evolution policy.

Use a separate PySpark notebook. Run the following cells in order. All `s04_` tables here are current-batch teaching outputs overwritten on rerun, not a permanent event history.

The source contract uses UTC timestamps with a trailing `Z`. The accepted measurement range is **-40.00 to 85.00 degrees Celsius**, inclusive, for this exercise. Real devices require an approved range and completeness policy.

The file also includes firmware and signal-strength context, plus export/UI/debug fields. They make the Silver column-selection exercise concrete. They do not change the event acceptance rules or expected business results.

Cell 1: Define context and read raw strings

PYTHON

```python
from pyspark.sql import functions as F, types as T, Window

BATCH_ID = "s04-devices-001"
INPUT = "Files/session_04/devices/batch-001/device_readings.csv"
BRONZE = "s04_device_events_bronze"
SILVER = "s04_device_events_silver"
REJECTED = "s04_device_events_rejected"
TRANSFER = "s04_device_daily_transfer"

spark.conf.set("spark.sql.session.timeZone", "UTC")

input_schema = T.StructType([
    T.StructField("device_id", T.StringType()),
    T.StructField("event_id", T.StringType()),
    T.StructField("event_time_utc", T.StringType()),
    T.StructField("temperature_c", T.StringType()),
    T.StructField("firmware_version", T.StringType()),
    T.StructField("signal_strength_dbm", T.StringType()),
    T.StructField("ui_theme", T.StringType()),
    T.StructField("debug_message", T.StringType()),
    T.StructField("export_row_number", T.StringType()),
])

raw = (
    spark.read.schema(input_schema)
    .option("header", True)
    .option("mode", "FAILFAST")
    .csv(INPUT)
    .withColumn("source_file", F.input_file_name())
    .withColumn("batch_id", F.lit(BATCH_ID))
    .withColumn("ingested_at_utc", F.current_timestamp())
)
(
    raw.write.format("delta").mode("overwrite")
    .option("overwriteSchema", "true").saveAsTable(BRONZE)
)
bronze = spark.table(BRONZE)

if bronze.count() != 9:
    raise ValueError("This fixture must contain nine raw device events.")
```

Inspect the source header and column order before execution. Retaining string fields avoids converting `sensor error` into a mysterious missing number at the landing boundary. The original source file remains in Files for replay.

Pause here for exploration and visualization and the Data Wrangler demonstration, then return to Cell 2. Those exercises do not overwrite Bronze or substitute a sampled result for full-batch validation.

Cell 2: Normalize keys, timestamp, and measurement

PYTHON

```python
def normalized_key(column):
    value = F.upper(F.trim(F.col(column)))
    return F.when(F.length(value) > 0, value)


prepared = (
    bronze.withColumn("device_id", normalized_key("device_id"))
    .withColumn("event_id", normalized_key("event_id"))
    .withColumn("event_time_text", F.trim("event_time_utc"))
    .withColumn("temperature_text", F.trim("temperature_c"))
    .withColumn(
        "event_time",
        F.expr("try_cast(event_time_text as timestamp)"),
    )
    .withColumn(
        "temperature",
        F.expr("try_cast(temperature_text as decimal(8,2))"),
    )
)

event_key = Window.partitionBy("batch_id", "device_id", "event_id")
evaluated = (
    prepared.withColumn("event_key_count", F.count("*").over(event_key))
    .withColumn(
        "quality_issue",
        F.when(
            F.col("device_id").isNull() | F.col("event_id").isNull(),
            "MISSING_EVENT_KEY",
        )
        .when(F.col("event_key_count") > 1, "DUPLICATE_EVENT")
        .when(
            F.col("event_time").isNull()
            | ~F.col("event_time_text").rlike(
                r"^[0-9]{4}-[0-9]{2}-[0-9]{2}T[0-9]{2}:[0-9]{2}:[0-9]{2}Z$"
            ),
            "INVALID_TIMESTAMP",
        )
        .when(
            F.col("temperature").isNull()
            | ~F.col("temperature_text").rlike(r"^-?[0-9]{1,3}(\.[0-9]{1,2})?$"),
            "INVALID_TEMPERATURE",
        )
```

```
        .when(
            ~F.col("temperature").between(-40, 85),
            "OUT_OF_RANGE",
        ),
    )
)
```

The timestamp format check rejects timezone-less text rather than silently treating local wall-clock time as UTC. The cast also rejects impossible/unparseable timestamps. Handling local zones and daylight-saving ambiguity is a separate source-contract decision.

Both copies of D01/E002 are quarantined. This intentionally conservative policy does not choose a first/last occurrence without an ordering contract.

Cell 3: Separate accepted and rejected events

PYTHON

```python
accepted = evaluated.filter(F.col("quality_issue").isNull())
rejected = evaluated.filter(F.col("quality_issue").isNotNull())

counts = (accepted.count(), rejected.count())
if counts != (4, 5):
    raise ValueError(f"Unexpected accepted/quarantine counts: {counts}")

silver_events = (
    accepted.select(
        "batch_id", "device_id", "event_id",
        F.col("event_time").alias("event_time_utc"),
        F.col("temperature").alias("temperature_c"),
        F.trim("firmware_version").alias("firmware_version"),
        F.expr("try_cast(signal_strength_dbm as int)").alias("signal_strength_dbm"),
        "source_file", "ingested_at_utc",
    )
    .withColumn("quality_rule_version", F.lit("s04-devices-v1"))
)
(
    silver_events.write.format("delta").mode("overwrite")
    .option("overwriteSchema", "true").saveAsTable(SILVER)
)
(
    rejected.write.format("delta").mode("overwrite")
    .option("overwriteSchema", "true").saveAsTable(REJECTED)
)

rejected.groupBy("quality_issue").count().show()
```

Expect two duplicate-event rows, one invalid temperature, one invalid timestamp, and one out-of-range reading. A malformed measurement is not a zero, and an outlier is not automatically corrected to the nearest allowed boundary.

The accepted Silver table has ten columns: typed event facts, firmware/signal context, and lineage. UI theme, debug text, export row position, raw text duplicates, and temporary helper columns are excluded from this contract, not deleted from raw evidence. Firmware and signal are optional diagnostic context; they do not determine temperature acceptance. All supplied signal values parse as integers. A production contract must define how missing/invalid optional context is flagged, not silently treat null as healthy signal.

Cell 4: Build the Warehouse handoff table

PYTHON

```python
silver_events = spark.table(SILVER)
daily = (
    silver_events.groupBy(
        "batch_id",
        F.to_date("event_time_utc").alias("reading_date"),
        "device_id",
    )
    .agg(
        F.count("*").alias("reading_count"),
        F.min("temperature_c").alias("min_temperature_c"),
        F.max("temperature_c").alias("max_temperature_c"),
        F.avg("temperature_c").cast("decimal(8,2)").alias("avg_temperature_c"),
    )
)

if daily.count() != 2:
    raise ValueError("Expected two device/day summaries.")
if daily.agg(F.sum("reading_count")).first()[0] != 4:
    raise ValueError("Daily summaries must account for four accepted events.")

daily.write.format("delta").mode("overwrite").saveAsTable(TRANSFER)
spark.table(TRANSFER).orderBy("device_id").show(truncate=False)
```

UTC date	Device	Accepted readings	Minimum C	Maximum C	Average C
2026-10-28	D01	2	20.00	24.00	22.00
2026-10-28	D02	2	30.00	32.00	31.00

The grain has changed: Gold will contain two device/day rows, not four accepted events and not nine raw events. Name the count as accepted readings so a consumer does not mistake it for complete device coverage.

The aggregation reads the persisted Silver contract, not a wide intermediate DataFrame. Firmware version and signal strength remain available in Silver for another approved use case; they are not arbitrarily added to the device/day grain. See Silver-to-Gold column design and the after-cleaning visualizations.

Discuss the production requirements not implemented here

This is batch processing. There are no streaming checkpoints, event-time watermarks, late-arrival corrections, or automatic reprocessing of previously published days.

For a real implementation, define whether an event key is unique across batches, how immutable raw input is retained, whether exact repeated events may be safely collapsed, which timestamp establishes a winner, and how corrections revise prior daily results. Never call `dropDuplicates()` a complete deduplication policy.

No collected event list is moved into Python driver memory; Spark performs the checks and aggregation. A different event source, workload size, or existing Warehouse layout could make a SQL-based alternative equally appropriate.

Demonstration and learner prompts

Predict the accepted count before running Cell 3. Ask whether removing the repeated E002 rows means all device activity is represented; the answer is no, and source completeness remains independent.

Have learners explain why UTC context affects the day grain and why an average over accepted readings needs a clear label. Then rerun unchanged input with no competing writer and compare business results, not ingestion timestamps.

Next: Publish the transfer table to Warehouse Gold.

Source: 03_device_telemetry.md. Companion-material links open the repository originals.